

BEYOND STATIC WRITES: TRAJECTORY AND STATE DIAGNOSTICS FOR IN-PLACE TEST-TIME TRAINING

Anonymous authors

Paper under double-blind review

ABSTRACT

Can changing where and when a language model writes at test time reveal utility that every constant write action misses? We study this question on a fixed Qwen3-1.7B In-Place TTT research checkpoint, using chunkwise action trajectories and answer-based diagnostic search. Across 6,000 RULER-16K examples, the observed dynamic lower bound exceeds the best constant action selected separately for each example by 1.522 pp. On the 1,435 non-ceiling examples where search actually runs, the score rises from 34.93 to 41.30: 6.362 pp, with a paired bootstrap 95% interval of [5.331, 7.433] pp. We then examine the 173 improved trajectories. Reversing their action multiset, resetting accumulated fast weights, or deleting a prescribed layer offset can reduce their scores. A continuation experiment finds effects of replacing the history-formed prefix cache even when fast weights are matched at the intervention boundary; execution paths also affect results in this implementation. Finally, exhaustive one-switch enumeration matches the final score of 60 of 73 improved native trajectories that originally use multiple switches. These conditional diagnostics separate observed sequential utility, sensitivity to carried state, and the switching complexity of score-matched alternatives. They do not provide an answer-free controller or establish an implementation-independent mechanism.

1 INTRODUCTION

Test-time training (TTT) lets a model adapt temporary parameters to its current input. In language modeling, fast weights can serve as an evolving state alongside attention activations and caches (Sun et al., 2024; Feng et al., 2026; Ouyang et al., 2026). Methods differ in their adaptation targets, writable parameters, and training procedures. Once these choices are fixed, a further question remains: how does the arrangement of writes within one input affect the utility that can be reached?

We study this question using a finite set of write actions on an In-Place TTT research checkpoint. A *constant trajectory* repeats one action throughout the input; a *dynamic trajectory* may change actions between chunks. Choosing the best constant trajectory separately for each example is already an answer-based oracle. Nevertheless, it cannot represent changes within an example. Our comparison asks whether a bounded search can find final scores outside this fully enumerated constant family. It does not assume that all prior TTT systems use uniform writes: adaptation selection and control are already studied in SCoL, GDWM, S-TTT, and SEAL (Wang et al., 2026; Mei et al., 2026; Zhu et al., 2026; Zweiger et al., 2025).

The denominator is central to this comparison. Of 6,000 examples in our dynamic evaluation scope, 4,565 already receive the maximum score under a constant action. We search the remaining 1,435 examples and retain the constant trajectories as candidates. Dynamic search increases the full-scope mean by 1.522 pp, but the increase is 6.362 pp on the examples where search actually runs. Reporting both makes the observed opportunity visible without replacing a population estimate by a success-selected one. The 173 examples that strictly improve are then frozen for separate diagnostic analyses.

Those analyses distinguish three questions. First, does a discovered trajectory retain its utility when the same actions are rearranged over the same input? Second, do replacements of the fast weights or prefix cache change a fixed continuation? Third, does matching a discovered score require many action switches? Reverse changes the assignment of actions to content chunks while preserving

their multiset. Reset and layer-offset Deletion replace carried weights while preserving the prefix cache and suffix actions. A factorial continuation experiment matches fast weights at a boundary and varies the cache source and execution path. Exhaustive one-switch enumeration then tests a restricted alternative family. Each procedure answers a different question; the intervention effects are not a causal decomposition of the search gain.

The study contributes (i) a quantified, reward-selected gap beyond per-example constant actions, including direct uncertainty estimates for the searched subset; (ii) paired schedule and state interventions with explicit cohort and execution semantics; and (iii) a separation between history sensitivity and score-level switching requirements. Among 73 improved native trajectories with multiple switches, 60 have an exactly-one-switch alternative with the same score. Our claims concern this checkpoint, action space, and recorded execution protocol. In particular, the runtime-path effects motivate numerical validation rather than a claim that a new learned memory has been discovered.

2 STUDY SETTING: TTT AS AN ACTION TRAJECTORY

2.1 RESEARCH CHECKPOINT AND WRITE RULE

Our implementation builds on the In-Place TTT framework (Feng et al., 2026), which adapts existing MLP down projections. Its next-position contextual target is informed by TTT-NTP (Ouyang et al., 2026). We use a direct outer-product update with a learned signed gate and norm clipping, rather than reproducing TTT-NTP’s complete inference procedure. The predictive target and in-place adaptation framework are background choices, not contributions of this study. Appendix A distinguishes this implementation from the referenced methods.

The checkpoint was initialized from Qwen3-1.7B Base (Yang et al., 2025) and continued through 297M long-text tokens followed by 10M question-answering sequence tokens, as recorded in the original experiment description. All evaluations use this same continued-trained checkpoint. OFF disables its test-time writes; it is not the original released Qwen checkpoint. The historical 1.7B training recipe and data anchors are not completely available in the source release, so these training quantities are reported provenance, not independently reproduced training runs (Appendix G.2).

The writable matrices are the MLP down projections in layers $\mathcal{L}_{\text{TTT}} = \{0, 6, 12, 18, 24\}$, with complete chunks of $C = 1024$ tokens. Let $z_{\ell,t,j}$ be the normalized MLP input after attention and its residual, and $u_{\ell,t,j}$ the gated MLP activation entering the down projection. With learned projection P_ℓ and signed channel vector g_ℓ , define

$$v_{\ell,t,j} = P_\ell^\top (g_\ell \odot z_{\ell,t,j+1}), \quad 1 \leq j < C, \quad (1)$$

$$D_{\ell,t} = \eta \sum_{j=1}^{C-1} v_{\ell,t,j} u_{\ell,t,j}^\top, \quad \Delta W_{\ell,t} = \text{clip}_{F,\epsilon}(D_{\ell,t}), \quad \eta = 1, \quad \epsilon = 10^{-5}. \quad (2)$$

Clipping limits each layer/chunk increment’s Frobenius norm; it is not an optimizer epsilon or an elementwise bound. There is no division by the number of pairs. The gate and projection are trained in the outer model and frozen during evaluation. The gate is neither a sigmoid nor an input-dependent action selector. Inference directly computes the update, with no inner loss backward pass or ridge-regression solve.

Each complete chunk first reads the effective weight available at its entrance and then produces its update. Pairs never cross chunk boundaries; the next-position representation is from the already observed input chunk. Incomplete tails and generated tokens read accumulated state without writing. The prompt forward pass proceeds layer by layer, with the chunk loop inside each writable MLP; it is not an online controller invoked once per chunk. Greedy decoding uses BF16, SDPA attention, and the benchmark’s explicit per-sample generation budget. Each independent trajectory starts from the same checkpoint. Downstream updates are recomputed rather than copied from a different trajectory.

2.2 ACTIONS, STATIC ORACLE, AND SEARCHED LOWER BOUND

For $T_x = \lfloor n_x / C \rfloor$ complete input chunks, a trajectory is $\pi = (a_1, \dots, a_{T_x})$, with

$$\mathcal{A} = \{\text{OFF}, L_0, L_6, L_{12}, L_{18}, L_{24}, \text{ALL}\}.$$

Scope	N	Sample Best	Dynamic LB	Gain (pp)	95% CI (pp)
Full 12-task scope	6,000	84.44	85.96	+1.522	[1.270, 1.788]
Searched non-ceiling	1,435	34.93	41.30	+6.362	[5.331, 7.433]
Improved only	173	40.48	93.26	+52.775	Descriptive subset

Table 1: Three distinct denominators. The searched subset is defined by the static ceiling, before examining dynamic success. Improved-only scores are conditional descriptions, not an estimate for all searched examples. Means use the 0–100 task-score scale; gains and CIs use unrounded paired values.

OFF suppresses the new write without clearing history; L_k writes only layer k , and ALL writes all five layers. The frozen task scorer returns $R(x, \pi) \in [0, 1]$; we report $100R$, a task score rather than uniformly calibrated accuracy. The per-example static oracle is

$$R_{\text{SB}}(x) = \max_{a \in \mathcal{A}} R(x, a^{T_x}). \quad (3)$$

DYNAMIC LB evaluates all seven constant seeds and performs one forward coordinate sweep. At each position it tries all seven actions for each current beam member, retaining a beam of four after expansion. Every unique candidate is evaluated by a clean full replay in the released implementation. All evaluated constants remain in the result pool, although not necessarily in the active beam. The final candidate comes from the observed maximum-score set, with ties preferring fewer switches, then fewer distinct actions, then a deterministic sequence hash. Beam ties use Hamming diversity (Appendix B).

This yields an observed lower bound on the best score over 7^{T_x} trajectories under this execution protocol. The final answer score selects the trajectory: it is a diagnostic oracle, not an answer-free policy. Likewise, nonnegative improvement over SAMPLE BEST follows from candidate retention. It is not evidence that an online controller cannot harm a sample.

2.3 EVALUATION SCOPE AND CONDITIONAL COHORTS

Fixed-action results cover 13 RULER-16K tasks with 500 examples each (Hsieh et al., 2024). Dynamic search covers 12 tasks, excluding CWE. The original allocation decision considered CWE’s low OFF performance (4.94 versus a SAMPLE BEST score of 5.24); it was not an outcome-independent task exclusion. We retain all 13 fixed-action results and describe a static-fallback aggregate separately in Appendix C.

In the 6,000-example dynamic scope, *ceiling* means $R_{\text{SB}} = 1$. Its 4,565 ceiling examples stay in the denominator with zero dynamic difference; the other 1,435 are searched. The 173 strict improvements form $\mathcal{I}$, used for Reverse and R2. A frozen early-switch, steady-tail rule selects 167 for Reset and continuation restoration; 151 have a deletable prefix component. We call each frozen winning trajectory NATIVE. These selected cohorts support conditional diagnostics, not population treatment effects. All revision confidence intervals are paired, 20,000-resample bootstraps; the full and non-ceiling search contrasts preserve task strata. Appendix B.3 specifies pairing, ordering, and the limits of these intervals.

3 SEQUENTIAL UTILITY BEYOND CONSTANT ACTIONS

Fixed writes have modest aggregate effects in this checkpoint. On all 13 tasks, OFF, the best globally fixed single layer (L_{24}), ALL, and SAMPLE BEST score 77.22, 77.32, 77.20, and 78.35, respectively. On the common 12-task scope these values are 83.24, 83.35, 83.21, and 84.44. The maximum in Equation 3 guarantees the static oracle’s advantage; that advantage alone does not establish stable sample-specific preferences. The relevant next comparison is whether a nonconstant trajectory can exceed every constant trajectory for the same example.

Table 1 shows a 1.522 pp full-scope gain and, more directly, a rise from 34.93 to 41.30 on the 1,435 examples actually searched: 6.362 pp, 95% CI [5.331, 7.433] pp. The latter is computed directly from the paired non-ceiling results. It is not merely an extrapolation from rounded population means.

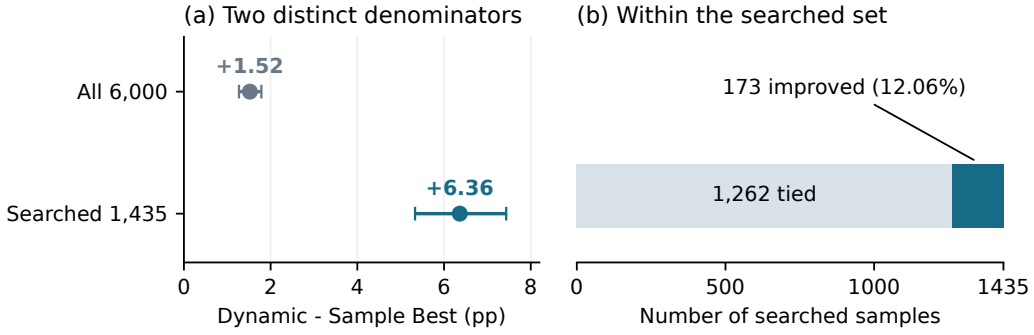


Figure 1: The searched-sample effect and its prevalence. Left: paired bootstrap 95% intervals on a common percentage-point scale. Right: 173 of the 1,435 searched examples improve, while 1,262 tie. The population gain includes 4,565 additional ceiling examples with zero difference.

The two estimates satisfy

$$\Delta_{\text{full}} = \frac{1435}{6000} \Delta_{\text{searched}},$$

because ceiling examples contribute zero by protocol. This identity describes denominator composition, not a causal decomposition.

Improvements occur on 12.06% of searched examples, or 2.88% of the full 12-task scope. The 173 successes gain 52.77 points on average, but this large number conditions on observed improvement and cannot replace the 6.36-point searched-set estimate. Nine tasks have non-ceiling examples and positive point estimates; three single-needle tasks are entirely at ceiling. Some conditional task estimates have very small denominators, so we do not claim significance for each task (Appendix C).

These results establish an observed gap beyond the seven constant trajectories in the recorded evaluator. They do not show that the search is efficient: actual candidate-count and latency records, and a matched-budget random search, are absent from the released result tables. The current code gives an upper bound of 289–385 unique candidate evaluations per searched example, not a measured cost. Numerical path validation is also needed before attributing every difference exclusively to write scheduling (Appendix G). With these boundaries, the frozen improvements provide a concrete set on which to study trajectory sensitivity.

4 SCHEDULE AND FAST-WEIGHT INTERVENTIONS

4.1 REARRANGING THE SAME ACTIONS OVER THE SAME CONTENT

For $\pi_x = (a_1, \dots, a_{T_x})$, Reverse evaluates $\pi_x^{\text{rev}} = (a_{T_x}, \dots, a_1)$ by recomputation. It preserves input order, action identities, their multiplicities, and the union of writable layers used by the trajectory. It changes which action is assigned to each content chunk and the state in which later writes occur. Thus it probes schedule–content alignment and history jointly, rather than isolating a content-independent ordering effect.

On the 173 improved examples, the score falls from 93.26 to 41.71, compared with a SAMPLE BEST score of 40.48 on the same examples (Table 2). Scores decrease on 163 examples and tie on 10. The action multiset alone therefore does not preserve these discovered outcomes. Since NATIVE was reward-selected, the large drop is not an unconditional causal effect or a proof that all TTT update operators are noncommutative. A random-permutation reference with the same action multiset would be needed to determine whether reversal is unusually disruptive among alternative arrangements.

4.2 RESETTING CARRIED WEIGHTS BEFORE A FIXED SUFFIX

Reset eligibility requires the first switched-to chunk to occur at or before 25% of trajectory length and one action to occupy at least 80% of the second half. This selects 167 examples. The inter-

Intervention	N	SB	Native	After	Drop	95% CI	$\downarrow / = / \uparrow$
Reverse	173	40.48	93.26	41.71	51.55	[46.53, 56.63]	163/10/0
Reset	167	41.22	93.01	49.15	43.86	[38.54, 49.36]	135/32/0
Reset, nontrivial	151	40.47	92.94	44.43	48.51	[43.12, 54.07]	135/16/0
Deletion	151	40.47	92.94	51.77	41.17	[36.52, 45.97]	134/17/0

Table 2: Paired interventions, with the same-cohort static oracle (SB). Scores are on the 0–100 scale; drops and intervals are in pp. Cohorts differ, so effect sizes should not be ranked across interventions. The nontrivial Reset cohort excludes the 16 all-OFF prefixes.

vention boundary b_x is separately defined as the first chunk of the *final contiguous constant-action suffix*, not necessarily the first switch or the start of the second half. Immediately before that chunk, Reset sets

$$W_{\ell, b_x^-}^{\text{Reset}} = W_{\ell}^{\text{ckpt}} \quad \text{for every } \ell \in \mathcal{L}_{\text{TTT}},$$

retains the native prefix attention cache and all suffix actions, and recomputes the continuation. The mean score falls from 93.01 to 49.15.

Sixteen eligible trajectories have an all-OFF prefix, so their Reset does not remove a previously written offset. All 16 have zero reward difference. Removing them by this structural criterion yields 151 examples and a 48.51-point mean drop (95% CI [43.12, 54.07]). We report both cohorts rather than counting unchanged all-OFF prefixes as positive evidence about fast-weight history.

4.3 SENSITIVITY TO A PRESCRIBED LAYER-OFFSET DELETION

Deletion removes $W_{\ell, b_x^-}^N - W_{\ell}^{\text{ckpt}}$ from a layer selected by a frozen prefix rule, preserving other boundary weights. Without an ALL action in the prefix, it targets the first written single layer. With ALL, it evaluates each of five layer deletions and averages their scores *within the sample*. The 151 eligible examples contribute 359 branches but only 151 statistical units. Their score falls from 92.94 to 51.77. The 99 available unwritten-layer sham replacements preserve predictions; these are zero-change mechanics controls, not magnitude-matched random perturbations. Accordingly, the result demonstrates sensitivity to this layer-offset replacement, without establishing that the deleted direction is uniquely responsible for the utility.

Reset and Deletion combine old prefix caches with replaced weights. They are deliberately mixed-state counterfactuals and do not mediate the Reverse drop. The natural all-OFF-prefix/native-suffix trajectory is a distinct comparator; its final score is not present in the released results. This limits how specifically the mixed-state losses can be interpreted.

5 CONTINUATION AT BOUNDARY-MATCHED FAST WEIGHTS

5.1 CACHE AND EXECUTION-PATH RESTORATION

Fast weights and attention caches can both depend on earlier adaptation. We ask whether a cache formed under a different write history changes the continuation after native fast weights have been restored at the boundary. For the same 167 eligible examples, define a control trajectory

$$\pi_x^C = (\underbrace{\text{OFF}, \dots, \text{OFF}}_{b_x - 1}, a_{b_x}^N, \dots, a_{T_x}^N).$$

It supplies a control prefix cache K^C ; K^N denotes the native cache. Here K is shorthand for *both* attention keys and values at all 28 layers. Only cache positions before the boundary are transplanted. The control trajectory is not the all-OFF baseline.

Every factorial branch starts the suffix from all five native effective weights W^N . The execution-path factor is P_0 , which explicitly materializes state for all five candidate layers, or P_1 , which follows the native materialization pattern and uses the standard MLP for unused layers. The four branches are

$$\begin{aligned} F_{00} &= (W^N, K^C, P_0), & F_{01} &= (W^N, K^C, P_1), \\ F_{10} &= (W^N, K^N, P_0), & F_{11} &= (W^N, K^N, P_1). \end{aligned} \quad (4)$$

Branch	Prefix cache	Runtime path	Exact closure	Mean score
F_{00}	K^C	P_0	97/167	72.73
F_{01}	K^C	P_1	108/167	77.13
F_{10}	K^N	P_0	151/167	87.23
F_{11}	K^N	P_1	167/167	93.01

Table 3: Restoration on 167 paired examples. All branches receive the native fast weights at the boundary. Exact closure tests the recorded continuation, and is stronger than equality of the final reward. Complete F_{11} restoration is a replay-integrity check.

These tuples specify conditions *at the intervention boundary*. All suffix activations, caches, new writes, and generation are recomputed. In particular, a cache replacement can change later fast weights; the contrast is a total downstream effect of the starting-cache intervention, not a direct effect measured with weights clamped throughout the suffix.

The recorded closure criterion checks the first continuation computation and update, later complete-chunk hidden states, the incomplete tail, generation-entry logits, and prediction hashes. Tensor checks use exact equality in the recorded execution environment. Restoring all recorded conditions closes 167/167 examples (Table 3); this verifies reconstruction of the reference, rather than establishing a new sufficient-state theorem.

5.2 PAIRED REWARD EFFECTS AND THEIR SCOPE

With the native runtime path, $F_{11} - F_{01}$ restores only the starting prefix cache. Its mean reward difference is 15.89 pp (95% CI [11.66, 20.49] pp): 52 examples improve and 115 tie. With the native cache, $F_{11} - F_{10}$ restores the runtime path and increases reward by 5.79 pp (95% CI [2.79, 9.18] pp): 15 improve and 152 tie. The first continuation computation and complete continuation chunks already close for all F_{10} examples; residual differences arise in the tail or generation-entry computation. Appendix E reports stagewise checks and paired statistics.

All four branches close for the 16 all-OFF prefixes identified in Section 4. Excluding these cases leaves cache and path reward contrasts of 17.57 and 6.40 points on 151 examples. Thus the paired effects are not created by including structurally unchanged prefixes.

The cache result concerns a history-formed cache under this intervention, not the general observation that Transformers use attention caches. The path result is implementation-specific. The evaluator selects its materialized layers from the entire planned trajectory, so different future actions can select different implementations even for an identical action prefix. The release does not establish numerical invariance between all such paths. Until those checks are available, P is an execution condition and a source of uncertainty in interpretation, not an additional learned memory. Neither reward agreement nor exact replay in one environment establishes a universal minimal state (W, K, P) .

6 ONE-SWITCH ALTERNATIVES TO IMPROVED TRAJECTORIES

History sensitivity does not determine how many switches a trajectory needs to reach a given final score. For each of the 173 improved examples, we exhaust the exactly-one-switch family

$$\Pi_{R2}(x) = \{A^\tau B^{T_x - \tau} : A, B \in \mathcal{A}, A \neq B, 1 \leq \tau < T_x\}, \quad R_{R2}(x) = \max_{\pi \in \Pi_{R2}(x)} R(x, \pi). \quad (5)$$

Both segments are nonempty and the switch does not reset accumulated state. There are $42(T_x - 1)$ candidates per example, 462–588 in this cohort, and 99,120 total. Exactness applies only to this restricted family. Selection still uses the final answer score.

The cohort means are 40.48 for SAMPLE BEST, 93.26 for NATIVE, and 88.76 for R2. R2 matches the native score on 160/173 examples and is lower on 13. The informative distinction is that 100 native trajectories already have one switch: recovering them is required by family inclusion under consistent execution. Among the remaining 73 multi-switch trajectories, **60/73 (82.19%) have an exactly-one-switch score-matched alternative**. Table 4 separates these cases.

Native trajectory subset	N	R2 matches	Native	R2
Native one switch	100	100	93.20	93.20
Native multiple switches	73	60	93.33	82.67
Multi-switch, full score	52	44	100.00	88.11
Multi-switch, below full score	21	16	76.83	69.21

Table 4: R2 recovery and ceiling sensitivity. The last two rows partition the multi-switch subset. Scores are means on the displayed cohort. The 100 already-one-switch matches are a consistency check; 60/73 is the main alternative-trajectory result.

No R2 candidate exceeds the native reward. Since 123/173 native rewards are already maximal, this is partly constrained by the score ceiling. Within the multi-switch subset, R2 matches 44/52 full-score and 16/21 non-full-score trajectories. Exhaustive failure on the remaining 13 establishes a gap for the zero- and one-switch families under the evaluated protocol; it does not certify global optimality of NATIVE.

Matching reward need not match generated text, fast weights, caches, or intermediate computation. The result is specific to 1024-token chunks and the seven-action space. A same-size random trajectory family has not been evaluated, so we make an existence claim about the one-switch alternatives, not a claim that this family is uniquely efficient or that an online controller can easily identify its best member.

7 RELATED WORK

TTT substrates and predictive targets. TTT for language models includes neighbor-based adaptation (Hardt & Sun, 2024), learned recurrent inner models (Sun et al., 2024), large-chunk learners (Zhang et al., 2025), in-place adaptation (Feng et al., 2026), and end-to-end long-context training (Tandon et al., 2025). In-Place TTT provides our implementation framework. TTT-NTP directly studies next-position contextual supervision (Ouyang et al., 2026), which informed the target used here. Our signed-gate outer-product implementation is not a replication of its regularized closed-form inference solver. We hold these design choices fixed and investigate trajectories and carried state.

Write selection and adaptation control. SCoL selects update locations while balancing incorporation and retention (Wang et al., 2026). GDWM allocates adaptation effort using a write controller under a budget (Mei et al., 2026). S-TTT selects evidence spans before adaptation (Zhu et al., 2026), and SEAL learns to generate self-edits and update directives using downstream feedback (Zweiger et al., 2025). These works already motivate selective adaptation. Our distinction is the combined diagnostic comparison of constant versus sequential actions, state replacements, and one-switch score matching. Forward-compute allocation, such as Mixture-of-Depths (Raposo et al., 2024), is related but does not by itself control temporary parameter writes.

Fast weights, caches, and interventions. The capacity of fast weights to retain history is established background (Ba et al., 2016; Schlag et al., 2021), not a novelty claim here. Analyses connecting KV-binding TTT to linear attention (Liu et al., 2026) concern associations within an inner learner; their keys and values should not be conflated with the Transformer prefix attention cache intervened on here. qTTT adapts query projections while retaining cached keys and values (Bansal et al., 2025); it supplies related context for coexisting cached representations and adapted parameters, but does not validate our mixed-state controls. Our replacements also relate to activation patching (Zhang & Nanda, 2024), whose interpretations depend on the corruption and readout metric. We therefore distinguish task reward, prediction identity, and exact trace closure, and restrict conclusions to the specified interventions.

8 DISCUSSION AND CONCLUSION

This study separates three observations. An answer-based search finds scores outside the constant-action family, with a 6.36-point gain on the examples actually searched. Successful trajectories can

be sensitive to their schedule and carried state. Yet many of their final scores can be reached by a trajectory with only one switch. These are complementary descriptions of the tested system, not evidence that sequential benefit requires a complex controller or that every TTT learner has the same mechanism.

The scope remains limited. Large intervention effects condition on dynamic success and subsequent structural screening. CWE was not searched. The checkpoint is continued-trained, and the small fixed-write gains do not establish improvement over the original Qwen model. The 16K evaluation does not demonstrate extension beyond the backbone’s native context window. Public code supports other model families and scales, but completed, aligned cross-model results are not available in this study. We consequently report no cross-model robustness claim.

Numerical execution and missing comparisons also constrain interpretation. Runtime paths affect some recorded outcomes; zero-write path equivalence and invariance to future action choices have not been established on the historical model. Matched-budget random search, random action permutations, a natural control continuation, and magnitude-matched nonzero deletion controls are not present in the released evidence. Our confidence intervals quantify variability in frozen result tables, not these uncertainties, training-seed variability, or search-selection bias. Appendix G states the checks needed to strengthen the mechanistic interpretation without treating proposed experiments as results.

Within these limits, the main finding is concrete: the strongest per-example constant choice still misses observed sequential utility, and 60/73 multi-switch improved trajectories admit an equally scoring one-switch alternative. Studying reachable utility, intervention sensitivity, and switching requirements separately makes those conclusions both visible and testable.

REPRODUCIBILITY STATEMENT

The supplementary package contains the frozen per-sample score tables, trajectory manifest, and CPU scripts generating this revision’s statistics and figures. Model execution additionally requires the historical checkpoint, benchmark inputs, and environment. Their availability and the limits of the public source snapshot are documented in Appendix G.2.

STATEMENT OF AI ASSISTANCE

AI assistance was used to revise the exposition, inspect code, and prepare statistical reanalysis and figures from existing experimental records. The revision does not add model-training or inference experiments.

REFERENCES

- Jimmy Ba, Geoffrey Hinton, Volodymyr Mnih, Joel Z. Leibo, and Catalin Ionescu. Using fast weights to attend to the recent past. In *Advances in Neural Information Processing Systems*, 2016. URL <https://arxiv.org/abs/1610.06258>.
- Rachit Bansal, Aston Zhang, Rishabh Tiwari, Lovish Madaan, Sai Surya Duvvuri, Devvrit Khatri, David Brandfonbrener, David Alvarez-Melis, Prajjwal Bhargava, Mihir Sanjay Kale, and Samy Jelassi. Let’s (not) just put things in context: Test-time training for long-context LLMs, 2025. URL <https://arxiv.org/abs/2512.13898>.
- Guhao Feng, Shengjie Luo, Kai Hua, Ge Zhang, Di He, Wenhao Huang, and Tianle Cai. In-place test-time training, 2026. URL <https://arxiv.org/abs/2604.06169>.
- Moritz Hardt and Yu Sun. Test-time training on nearest neighbors for large language models. In *International Conference on Learning Representations*, 2024. URL <https://arxiv.org/abs/2305.18466>.
- Cheng-Ping Hsieh, Simeng Sun, Samuel Kriman, Shantanu Acharya, Dima Rekish, Fei Jia, Yang Zhang, and Boris Ginsburg. RULER: What’s the real context size of your long-context language models?, 2024. URL <https://arxiv.org/abs/2404.06654>.
- Junchen Liu, Sven Elfle, Or Litany, Zan Gojcic, and Ruilong Li. Test-time training with KV binding is secretly linear attention, 2026. URL <https://arxiv.org/abs/2602.21204>.
- Lingrui Mei, Shenghua Liu, Yiwei Wang, Yuyao Ge, Baolong Bi, Jiayu Yao, Jun Wan, Ziling Yin, Jiafeng Guo, and Xueqi Cheng. Gated differentiable working memory for long-context language modeling. In *Proceedings of the 64th Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pp. 31885–31913. Association for Computational Linguistics, 2026. doi: 10.18653/v1/2026.acl-long.1471. URL <https://aclanthology.org/2026.acl-long.1471/>.
- Xuan Ouyang, Zefan Cai, and Junjie Hu. Test-time training with next-token prediction, 2026. URL <https://arxiv.org/abs/2606.21803>.
- David Raposo, Sam Ritter, Blake Richards, Timothy Lillicrap, Peter Conway Humphreys, and Adam Santoro. Mixture-of-depths: Dynamically allocating compute in transformer-based language models, 2024. URL <https://arxiv.org/abs/2404.02258>.
- Imanol Schlag, Kazuki Irie, and Jürgen Schmidhuber. Linear transformers are secretly fast weight programmers. In *International Conference on Machine Learning*, 2021. URL <https://arxiv.org/abs/2102.11174>.
- Yu Sun, Xinhao Li, Karan Dalal, Jiarui Xu, Arjun Vikram, Genghan Zhang, Yann Dubois, Xinlei Chen, Xiaolong Wang, Sanmi Koyejo, Tatsunori Hashimoto, and Carlos Guestrin. Learning to (learn at test time): RNNs with expressive hidden states, 2024. URL <https://arxiv.org/abs/2407.04620>.
- Arnub Tandon, Karan Dalal, Xinhao Li, Daniel Kocaja, Marcel Rød, Sam Buchanan, Xiaolong Wang, Jure Leskovec, Sanmi Koyejo, Tatsunori Hashimoto, Carlos Guestrin, Jed McCaleb, Yejin Choi, and Yu Sun. End-to-end test-time training for long context, 2025. URL <https://arxiv.org/abs/2512.23675>.
- Zekun Wang, Anant Gupta, Zihan Dong, and Christopher J. MacLellan. Self-consolidating language models: Continual knowledge incorporation from context, 2026. URL <https://arxiv.org/abs/2605.07076>.
- An Yang, Anfeng Li, Baosong Yang, Beichen Zhang, Binyuan Hui, Bo Zheng, Bowen Yu, Chang Gao, Chengen Huang, Chenxu Lv, Chujie Zheng, Dayiheng Liu, Fan Zhou, Fei Huang, Feng Hu, Hao Ge, Haoran Wei, Huan Lin, Jialong Tang, Jian Yang, Jianhong Tu, Jianwei Zhang, Jianxin Yang, Jiayi Yang, Jing Zhou, Jingren Zhou, Junyang Lin, Kai Dang, Keqin Bao, Kexin Yang, Le Yu, Lianghao Deng, Mei Li, Mingfeng Xue, Mingze Li, Pei Zhang, Peng Wang, Qin Zhu, Rui Men, Ruize Gao, Shixuan Liu, Shuang Luo, Tianhao Li, Tianyi Tang, Wenbiao Yin, Xingzhang

Ren, Xinyu Wang, Xinyu Zhang, Xuancheng Ren, Yang Fan, Yang Su, Yichang Zhang, Yinger Zhang, Yu Wan, Yuqiong Liu, Zekun Wang, Zeyu Cui, Zhenru Zhang, Zhipeng Zhou, and Zihan Qiu. Qwen3 technical report, 2025. URL <https://arxiv.org/abs/2505.09388>.

Fred Zhang and Neel Nanda. Towards best practices of activation patching in language models: Metrics and methods. In *International Conference on Learning Representations*, 2024. URL <https://arxiv.org/abs/2309.16042>.

Tianyuan Zhang, Sai Bi, Yicong Hong, Kai Zhang, Fujun Luan, Songlin Yang, Kalyan Sunkavalli, William T. Freeman, and Hao Tan. Test-time training done right, 2025. URL <https://arxiv.org/abs/2505.23884>.

Xinyu Zhu, Zhe Xu, Xiaohan Wei, Yunchen Pu, Fei Tian, Chonglin Sun, Kaushik Rangadurai, Hua Zhi, Frank Shyu, Sandeep Pandey, Luke Simon, Yu Meng, and Xi Liu. Self-guided test-time training for long-context LLMs, 2026. URL <https://arxiv.org/abs/2607.09415>.

Adam Zweiger, Jyothish Pari, Han Guo, Ekin Akyürek, Yoon Kim, and Pulkit Agrawal. Self-adapting language models, 2025. URL <https://arxiv.org/abs/2506.10943>.

A IMPLEMENTATION AND RELATION TO PRIOR WORK

A.1 WHAT IS INHERITED AND WHAT IS STUDIED

Our study uses the in-place adaptation framework of Feng et al. (2026) and a next-position contextual target informed by Ouyang et al. (2026). This attribution separates an implementation foundation from a target-design influence. We do not claim to originate the predictive target, the use of down projections as fast weights, or the fact that fast weights retain history. The study fixes these choices to investigate trajectory utility, interventions, and score-matched alternatives. Development chronology and publication acceptance dates do not change these technical relationships.

Aspect	Referenced work	Current research implementation
Adaptation framework	In-Place TTT reuses MLP down projections and chunkwise writes.	Five fixed candidate down projections, evaluated with seven discrete write actions.
Target	TTT-NTP pairs a current gated MLP activation with a next-position contextual representation.	Next-position target multiplied by a learned signed channel vector and transformed by a learned projection.
Inference write	TTT-NTP describes a regularized closed-form inference solve.	Direct summed outer products, clipped per layer and chunk; no ridge solve or inner backward pass.
Question	The references develop the adaptation framework and predictive supervision.	The checkpoint and write rule are fixed while trajectories and carried state are varied.

Table 5: Technical relationships, not a claim that all parts of the present implementation were copied from either reference. The inference rule is distinguished from outer-training objectives.

A.2 DIMENSIONS, GATE, AND TARGET CONSTRUCTION

Let d denote the model hidden dimension and m the MLP intermediate dimension. Using column-vector notation, $z_{\ell,t,j} \in \mathbb{R}^d$, $u_{\ell,t,j} \in \mathbb{R}^m$, $g_\ell \in \mathbb{R}^d$, and $P_\ell \in \mathbb{R}^{d \times d}$. The writable down projection is $W_\ell \in \mathbb{R}^{d \times m}$, so $v_{\ell,t,j} u_{\ell,t,j}^\top$ has the correct weight shape. The MLP feature is

$$u = \text{SiLU}(W_{\text{gate}} z) \odot (W_{\text{up}} z).$$

This architectural gate is distinct from the learned target channel vector g_ℓ . The latter is signed, initialized at zero in the model definition, and learned during outer training. It is frozen, but not assumed zero, at evaluation. The target projection is likewise learned and then frozen.

The paired positions are $j = 1, \dots, C - 1$, with target $P_\ell^\top (g_\ell \odot z_{\ell,t,j+1})$. Neither a target from the next chunk nor an unobserved generated token is used. No mean normalization divides the sum by $C - 1$. Although the framework retains a convolution-target implementation for other configurations, the historical setting analyzed here selects the gated next-position target.

For an active layer/chunk, the implemented norm operation corresponds to

$$\text{clip}_{F,\epsilon}(D) = D \min \left(1, \frac{\epsilon}{\max(\|D\|_F, \delta_{\text{tiny}})} \right).$$

The norm is evaluated in float32, with its smallest positive normal value used to protect the denominator; the scale is cast back to the update dtype. The effective weight becomes $W_{\ell,t+1} = W_{\ell,t} + \Delta W_{\ell,t}$. The BF16 addition may round entries, so the norm of a proposed clipped increment should be distinguished from $\|f(W + \Delta W) - W\|_F$, the actually applied change. The latter matters for numerical audits and nonzero perturbation matching.

A.3 EXECUTION AND SAMPLE ISOLATION

The historical inference configuration specifies 16,384-token context, 1024-token chunks, layers 0/6/12/18/24, learning rate 1, and per-increment clip norm 10^{-5} . It uses BF16 and SDPA, one example per TTT forward, greedy decoding, tokenizer EOS, and a benchmark-specified maximum generation length for each sample. Tail and generation updates are disabled.

The prompt advances through Transformer layers. Inside each writable MLP, the complete chunks read and update their own effective weight sequentially. Consequently, the chunk action sequence is known before this layer-major forward. The write at a layer/chunk does not retroactively alter that same layer’s chunk output. It can affect later computation that reads the updated weight. A trajectory is not a token-level online policy, and the final answer is used for scoring and selection, not as the input to the target in Equation 1.

The effective weights are temporary per-trajectory state. The checkpoint parameters are not overwritten by these diagnostic evaluations. The released evaluator checks checkpoint-weight hashes after evaluation. It also compares constant-trajectory predictions against fixed-action records and forces a second capture of the selected winner. These are useful implemented gates; they are not substitutes for an independent historical GPU replay in this revision.

A.4 IMPLEMENTATION MAP

The companion provenance file pins a single code snapshot. Relative source paths below identify the definitions used for this revision.

Definition	Source within the repository
Gated target and weight update	src/dynamic_ttt/ttt/qwen_state.py
Search and candidate replay	src/dynamic_ttt/dynamic/search.py
Inference parameters	configs/inference/qwen3_1p7b.json
Eligibility and branch plans	src/dynamic_ttt/mechanisms/runtime.py
Reset/Deletion/KP execution	src/dynamic_ttt/mechanisms/execution.py
Trace comparisons	src/dynamic_ttt/mechanisms/kernels/trace.py

Table 6: A map from methodological definitions to inspectable source. Source availability does not imply availability of all model and dataset assets.

B SEARCH AND STATISTICAL PROTOCOL

B.1 ONE FORWARD COORDINATE SWEEP

The following procedure describes the released search. Its active beam and its pool of all evaluated candidates are distinct.

1. Evaluate all seven constant trajectories from a fresh checkpoint state. Retain their scores and prediction hashes. Check their consistency with the corresponding fixed-action records.
2. Initialize the active beam with all seven seeds and the result pool with those evaluations.
3. Visit chunk positions $1, \dots, T_x$ once in forward order. At each position, replace that coordinate of each active beam member by each of the seven actions. Reuse cached records only for identical complete trajectories; evaluate every new complete trajectory with a clean replay.
4. Add every expanded candidate to the result pool. Sort scores in descending order and retain up to four beam members. Among exact score ties, prefer a candidate with larger minimum Hamming distance to already selected members, then the ascending canonical sequence hash.
5. In the result pool, form the set within 10^{-12} of the best observed score. Select the candidate with fewest switches, then fewest distinct actions, then ascending sequence hash. Force a fresh winner capture and check its reward and prediction hash against the selected record.

The hash is SHA256 of the canonical JSON action sequence. Static seeds remain in the pool even when they leave the active beam, giving the static lower-bound guarantee up to the score-comparison tolerance. The final few-switch tie preference also matters when interpreting native trajectory structure. The public winning-trajectory manifest does not retain all losing candidates, so it is insufficient to independently reconstruct every historical tie decision.

B.2 CANDIDATE ACCOUNTING AND THE MEANING OF A BUDGET

There are seven seed evaluations. At the first coordinate, each seed has at most six new alternatives. At each subsequent coordinate, each of four beam members has at most six new alternatives. Deduplication can reduce these counts. Thus the number of unique candidate evaluations is bounded by

$$Q_{\max}(T) = 7 + 7(7 - 1) + (T - 1)4(7 - 1) = 24T + 25. \quad (6)$$

The released searched trajectories have $T \in \{11, 12, 13, 14, 15\}$, with counts 4, 110, 128, 256, and 937 respectively. The per-example upper bounds are 289–385, summing to 531,883 over the 1,435 searched examples. The final forced capture is a separate replay for each winner. These figures are derived upper bounds, not observed GPU calls, latency, or memory cost; fixed-action evaluation is another workload.

The full execution JSON schema contains candidate counts and timing fields, but the released winner-only JSONL does not. A measured cost comparison requires those original candidate records, including retries and forced captures. Beam width alone is not a compute budget.

With deterministic execution, complete enumeration of the seven static trajectories and replay of a better nonconstant one can establish a within-evaluator existence gap. It does not require pretending that hundreds of repeated identical static evaluations add new candidates. However, stochastic or numerical variation can interact with selection, and a larger search cannot be called more efficient merely because it finds a better score. A budget-matched random trajectory search is required for that additional comparison. It has not been run in the evidence supplied here.

B.3 ESTIMANDS, PAIRING, AND REPRODUCIBLE INTERVALS

Let $d_x = 100(R_{\text{Dyn}}(x) - R_{\text{SB}}(x))$. The full-scope estimand is the mean of d_x over all 6,000 examples, including ceiling zero differences. The searched estimand is the mean over $\{x : R_{\text{SB}}(x) < 1\}$, with $N = 1,435$. Although the full tasks are equally sized, the searched tasks are not. We preserve their observed counts and average over examples; we do not switch to an equally weighted mean of task means. Tasks with no searched examples contribute no observations to this conditional estimate.

For these two contrasts we resample sample pairs within each task, preserving the observed task sample sizes. For Reverse, Reset, Deletion, and K/P contrasts, we resample paired examples in the frozen selected cohort. Deletion scores are already averages over all eligible branches for that example. K/P scores are first pivoted by sample ID and differenced within the same example; four branch rows do not constitute four independent units. The sensitivity analyses use the structurally defined nontrivial-prefix cohort, without selecting on intervention reward.

Every interval in this revision uses 20,000 bootstrap draws, NumPy’s default random generator with seed 20260913 restarted for each contrast, and the 2.5th and 97.5th percentiles. Search strata are traversed in lexicographic task order, retaining original CSV order inside each task. Other mechanism contrasts retain their CSV order; K/P uses sorted sample IDs after pivoting. The input hashes and software versions are recorded with the output. This fully specified ordering makes the displayed endpoints reproducible. No multiplicity adjustment is applied; these are descriptive, paired uncertainty summaries for prespecified contrasts in a post-hoc revision.

The revised population interval is [1.270, 1.788] pp, and the searched interval is [5.331, 7.433] pp. The old manuscript reported a population interval of [1.267, 1.783] using a different finite-bootstrap record. The current release’s historical 10,000-draw procedure yields approximately [1.2686, 1.7858] in its CSV row order. We use the consistently regenerated revision intervals throughout instead of mixing historical and new endpoints. Point estimates are unchanged.

These intervals condition on the trained checkpoint, task sample composition, recorded search output, and cohort selection. They do not characterize retraining, repeated search randomness, GPU kernel variation, runtime-path equivalence, or the bias introduced by selecting successful trajectories. The improved-only 52.77-point gain is presented descriptively for this reason.

Task	OFF	L_0	L_6	L_{12}	L_{18}	L_{24}	ALL	SB
CWE	4.94	5.00	5.02	5.02	5.04	4.96	5.00	5.24
FWE	76.93	76.93	76.80	77.00	76.87	77.40	77.07	78.27
NIAH Single 1	100.00	100.00	100.00	100.00	100.00	100.00	100.00	100.00
NIAH Single 2	100.00	100.00	100.00	100.00	100.00	100.00	100.00	100.00
NIAH Single 3	100.00	100.00	100.00	100.00	100.00	100.00	100.00	100.00
NIAH Multikey 1	98.60	98.60	98.40	98.60	98.80	98.60	98.20	98.80
NIAH Multikey 2	95.20	95.20	95.20	95.20	95.20	95.20	95.20	95.40
NIAH Multikey 3	71.00	70.60	70.40	70.60	70.80	71.40	71.00	73.60
NIAH Multivalue	98.35	98.35	98.25	98.45	98.10	98.35	98.45	98.85
NIAH Multiquery	96.30	96.55	97.10	96.80	96.55	96.30	96.75	97.70
SQuAD	50.60	51.20	51.40	50.40	50.00	50.80	50.40	52.80
HotpotQA	41.80	41.60	42.60	42.40	42.40	42.00	41.60	44.00
VT	70.12	68.80	69.80	69.56	70.08	70.12	69.88	73.84

Table 7: All seven constant actions on the full 13-task evaluation. Each task has 500 examples. SB is the mean of per-example maxima, not the maximum of the displayed task means. Scores are 100R.

Task	Searched	Improved	SB	Dynamic	Full gain	Searched gain
FWE	312	24	78.27	79.87	1.60	2.56
NIAH Multikey 1	6	1	98.80	99.00	0.20	16.67
NIAH Multikey 2	23	4	95.40	96.20	0.80	17.39
NIAH Multikey 3	132	14	73.60	76.40	2.80	10.61
NIAH Multiquery	44	12	97.70	98.35	0.65	7.39
NIAH Multivalue	21	8	98.85	99.30	0.45	10.71
NIAH Single 1	0	0	100.00	100.00	0.00	–
NIAH Single 2	0	0	100.00	100.00	0.00	–
NIAH Single 3	0	0	100.00	100.00	0.00	–
HotpotQA	280	18	44.00	47.60	3.60	6.43
SQuAD	236	15	52.80	55.80	3.00	6.36
VT	381	77	73.84	79.00	5.16	6.77

Table 8: Dynamic results by task. SB and Dynamic use all 500 examples in that task. Gains are in pp; the last column uses only the number of searched examples shown. A dash means no non-ceiling examples, not an estimated zero conditional effect.

C COMPLETE FIXED RESULTS AND DYNAMIC DENOMINATORS

C.1 FIXED VERSUS SEARCHED SCOPE

Across the full 6,500 examples, the static oracle scores 78.35. The 12-task dynamic scope has a higher baseline because it excludes CWE, not because the same observations improve simply by changing the denominator. CWE remains in Table 7. Its low OFF score informed the historical search allocation, so dynamic claims are explicitly restricted to the searched task scope.

A descriptive 13-task hybrid can use Dynamic on the 12 evaluated tasks and SAMPLE BEST on CWE. Its gain over the full static oracle is

$$\frac{6000}{6500} \times 1.5216667 = 1.4046154 \text{ pp.}$$

This is a *searched-12 plus static-fallback-on-CWE* protocol, not a completed 13-task Dynamic experiment or evidence that CWE has no dynamic opportunity. We do not assign the unmeasured CWE dynamic outcome a scientific zero effect.

C.2 TASK VARIATION AND COHORT LINEAGE

Figure 2 displays conditional task point estimates with their actual denominators. A gain of 16.67 points for Multikey 1 comes from only six searched examples; it should not be compared as equally precise to the estimate on 381 searched VT examples. No taskwise confidence or significance claim is inferred from the pooled interval.

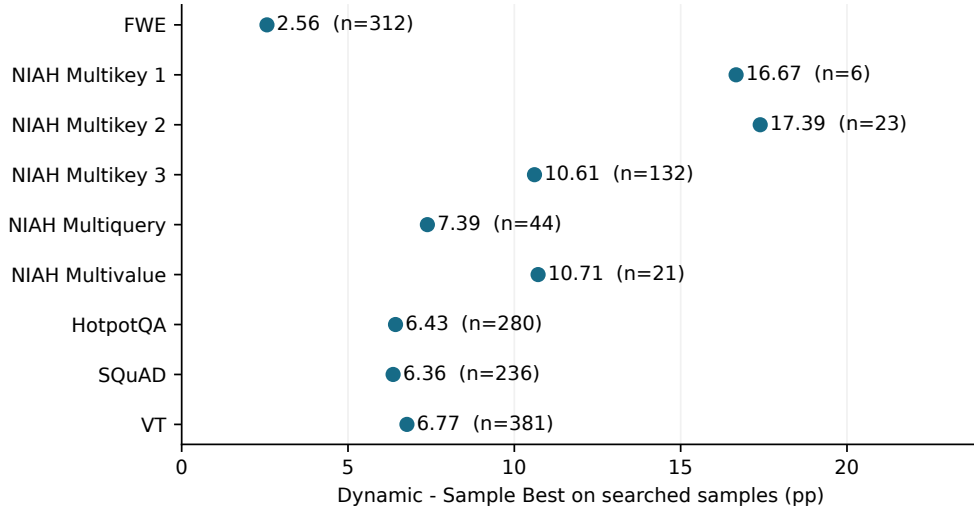


Figure 2: Searched-sample point estimates. Labels report the actual searched sample size. Three all-ceiling tasks have no conditional estimate and are omitted from this plot.

Cohort	N	Definition and downstream use
Fixed actions	6,500	All 13 tasks, 500 examples each.
Dynamic scope	6,000	The 12 tasks excluding CWE; includes ceiling examples.
Searched	1,435	Static oracle below score 1.
Improved	173	Dynamic strictly exceeds the static oracle; Reverse and R2.
Early-steady	167	Frozen structural screening of improved trajectories; Reset and K/P restoration.
Nontrivial prefix	151	At least one writable prefix component; Deletion and sensitivity analyses of Reset and K/P.

Table 9: The interventions branch from frozen cohorts. They are not a chain of treatments successively applied to the same resulting state.

D SCHEDULE, RESET, AND DELETION DETAILS

D.1 ELIGIBILITY AND INTERVENTION BOUNDARY

Write indices are one-based. Let $s = \min\{i \in \{2, \dots, T\} : a_i \neq a_{i-1}\}$. The early-switch test is $s/T \leq 0.25$. For the second-half sequence $(a_{\lfloor T/2 \rfloor + 1}, \dots, a_T)$, the highest-frequency action must account for at least 80% of its entries. Frequency ties use the fixed action order. The rule was frozen for the original mechanism cohort; it should not be interpreted as a population-wide sampling design.

The intervention boundary is defined separately:

$$b = 1 + \max\{i < T : a_i \neq a_T\}.$$

Equivalently, it is the start of the final contiguous run of a_T . Early switching and a dominant second-half action do not require all of the second half to be constant. Therefore the first switch, the half-way point, and b should not be conflated. The boundary ranges from 2 to 14. The counts for $b = 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 14$ are respectively 88, 25, 15, 14, 4, 9, 3, 3, 1, 4, and 1; no examples have $b = 12$ or 13.

D.2 WHAT EACH INTERVENTION PRESERVES

Reverse swaps the sequence of actions, not a stored sequence of weight increments. An increment depends on the activations and state at which it is computed. Copying native increments into a reversed schedule would therefore define a different experiment.

Intervention	Preserved	Changed and recomputed
Reverse	Input and its order, action multiset, length, and union of used layers.	Action-to-content assignment and the resulting state evolution. All counterfactual writes are recomputed.
Reset	Native prefix cache, suffix actions, input, and generation settings.	All five effective weights are reset at b^- ; later activations, weights, cache entries, and predictions are recomputed.
Deletion	Native prefix cache, suffix actions, and untargeted boundary weights.	One prescribed layer offset is removed per branch; all resulting downstream changes are recomputed.

Table 10: Intervention semantics. Preserving suffix actions does not preserve suffix activations, weight updates, or final predictions.

For Reset, maintaining the old cache while restoring weights is an intentional state replacement. It does not reconstruct the state that naturally arises from never writing the prefix. This is why the final reward of the natural π^C trajectory would be an additional informative comparison. Its prefix is already constructed as a K/P donor, but the released reward tables do not supply its complete continuation result.

D.3 DELETION TARGETS AND STATISTICAL UNIT

For a native prefix $a_1, \dots, a_{b-1}$, the target-selection rule uses actions rather than measured reward drops. If the prefix has no ALL, the first non-OFF single-layer action determines the deleted component. If it has ALL, each candidate layer is deleted in a separate branch. The deleted state is the accumulated layer offset at the boundary, not one historical write event.

There are 151 eligible samples, 359 branches, and a mean branch-averaged score of 51.77. For each example x , with prescribed target set J_x ,

$$\bar{R}_{\text{Del}}(x) = \frac{1}{|J_x|} \sum_{\ell \in J_x} R_{\text{Del},\ell}(x).$$

The reported drop averages $100(R_N(x) - \bar{R}_{\text{Del}}(x))$ over examples. Treating the 359 branches as independent would overweight examples containing an ALL prefix.

The 99 unwritten-layer shams check same-value replacement. They do not compare the deleted historical direction with arbitrary nonzero changes. A stronger direction-specific conclusion requires same-layer perturbations matched to the *actually applied* weight-change norm after rounding, using directions and seeds chosen without observing their effects.

D.4 NONTRIVIAL-PREFIX SENSITIVITY

The all-OFF-prefix indicator is computed from the native action sequence before b , without reading intervention outcomes. It identifies 16 examples in the 167-example Reset/KP cohort. Every one has a zero Reset reward difference and full closure under every K/P branch. Their removal leaves exactly the 151 sample IDs in the Deletion cohort. The IDs are listed in the generated analysis summary.

Reset on this cohort scores 44.43 against native 92.94, a paired drop of 48.51 points with interval [43.12, 54.07]. There are 135 decreases, 16 ties, and no increases. The corresponding Deletion score is 51.77. These two means now refer to identical sample IDs, but they remain effects of different replacement operations, not a decomposition into independent causal components.

E CONTINUATION RESTORATION AND PAIRED REWARD ANALYSIS

E.1 DONOR STATES AND EXECUTION PATHS

Both native and control donors see the same input tokens. The control sets all prefix actions to OFF and retains the native suffix actions. At boundary b , all branches receive native effective fast weights

for the five candidate layers. The cache factor selects control or native prefix keys and values at every attention layer. Only the first $(b - 1)C$ positions are eligible for transplantation; no native future cache entry is reused.

Let $U(\pi)$ be the union of layers written anywhere in the complete native trajectory. Path P_1 uses this native materialization pattern; a candidate layer outside the union uses the standard MLP path. P_0 materializes all five candidate states. This is an execution-path intervention, not a different learned write controller. Because the union uses the complete trajectory, the path choice is not determined solely by past actions. Appendix G discusses the consequent invariance question.

At injection, weight equality is checked. Thereafter, the branches recompute their own writes. In the cache contrast, equal boundary W and path do not imply equal later W . Calling this experiment a direct cache effect at globally fixed weights would change its estimand.

E.2 WHAT EXACT CLOSURE MEANS

The audit uses six named checks. They are not all cumulative:

1. L1: exact agreement in the first continuation chunk and first post-boundary update at the recorded sites.
2. L2: exact agreement in the recorded hidden states of complete continuation chunks.
3. L3: exact agreement in the incomplete input-tail trace.
4. L4: exact equality of generation-entry logits.
5. L5: equality of the decoded prediction hash.
6. L6: the conjunction of L1–L5.

For example, prediction agreement can hold when the tensor trace differs. This explains why an L5 count may exceed an earlier tensor-level count. Reward agreement is still weaker than prediction identity and is reported separately.

Cell	L1	L2	L3	L4	L5	L6	Mean logit distance
F_{00}	108	108	97	97	105	97	0.0022048
F_{01}	108	108	108	108	115	108	0.0017756
F_{10}	167	167	151	151	152	151	0.0006171
F_{11}	167	167	167	167	167	167	0.0000000

Table 11: Counts out of 167 at each closure check. Logit distance is relative ℓ_2 distance at generation entry. Exact equality is conditional on the recorded hardware, kernels, dtype, and trace definition; it is not a hardware-independent invariance statement.

For F_{10} , L1 and L2 pass for all 167 examples, while 151 pass the tail/logit trace and 152 share the decoded prediction. The complete F_{11} condition restores the trace and prediction on all examples. The scientific comparisons use the partially restored branches; the full reconstruction also checks implementation consistency.

E.3 REWARD CONTRASTS AND SENSITIVITY ANALYSIS

For each example, define

$$d_K(x) = 100(R_{11}(x) - R_{01}(x)), \quad d_P(x) = 100(R_{11}(x) - R_{10}(x)).$$

These contrasts differ only in the designated starting condition, with later changes allowed to propagate. We bootstrap these within-example differences. We do not substitute confidence intervals for logit-distance effects into a table of reward effects.

The cache restoration improves 52 of 167 rewards, although 59 examples gain full trace closure between F_{01} and F_{11} . Similarly, 15 rewards improve while 16 examples gain closure between F_{10} and F_{11} . These differences demonstrate why trace identity and task utility should be reported as separate criteria.

Cohort and contrast	N	Mean gain (pp)	95% CI (pp)	+ / 0 / -
All: $F_{11} - F_{01}$	167	15.89	[11.66, 20.49]	52/115/0
All: $F_{11} - F_{10}$	167	5.79	[2.79, 9.18]	15/152/0
Nontrivial: $F_{11} - F_{01}$	151	17.57	[12.96, 22.45]	52/99/0
Nontrivial: $F_{11} - F_{10}$	151	6.40	[3.31, 10.15]	15/136/0

Table 12: Paired reward effects. The nontrivial cohort excludes all-OFF prefixes by structure. All four contrasts use the same resampling protocol described in Appendix B.3.

Cell	Exact closure	Mean score	Mean logit distance
F_{00}	81/151	70.51	0.0024384
F_{01}	92/151	75.36	0.0019638
F_{10}	135/151	86.53	0.0006825
F_{11}	151/151	92.94	0.0000000

Table 13: K/P results on the 151 nontrivial-prefix examples. The 16 removed examples close under every cell, so subtracting them from the full counts does not create new mismatches.

F ONE-SWITCH ENUMERATION AND ITS LIMITS

Each ordered action pair $A \neq B$ contributes $T - 1$ switch positions. The family has exactly $7 \cdot 6 \cdot (T - 1)$ members, with state carried across the switch. R2 evaluates the complete family, not a sampled shortlist. The public per-example candidate counts sum to 99,120. A candidate is selected using the same task reward, so R2 is an oracle about attainable scores within this family and does not solve online action selection.

The 100 native trajectories with one switch are included in R2 by construction. Their recovery checks that family inclusion is reflected in execution and scoring. The nontrivial structural result concerns the other 73 native trajectories: 60 have a one-switch match and 13 do not. For these 13, the best one-switch reward equals the static oracle reward in the released table. Since all 173 native rewards exceed that oracle, these 13 examples retain a positive gap to both the zero-switch and exactly-one-switch families.

There are 50 native examples below the maximum task score and 123 at the maximum. Within the multi-switch group the corresponding counts are 21 and 52. One-switch matches are found for 16/21 and 44/52 respectively. Thus the 82.19% multi-switch match rate is not solely a full-score subset statistic, although ceiling effects restrict how often R2 could exceed native rewards. No candidate exceeding native was observed; the full dynamic optimum was not enumerated.

The average native-minus-R2 difference is 4.50 points. The aggregate gain retention relative to OFF, if desired as a descriptive summary, is

$$\text{Ret}_{\text{OFF}} = \frac{\sum_{x \in \mathcal{I}} (R_{\text{R2}}(x) - R_{\text{OFF}}(x))}{\sum_{x \in \mathcal{I}} (R_N(x) - R_{\text{OFF}}(x))} = 91.66\%.$$

It is a ratio of sums, not a mean of per-sample ratios, and its reference is OFF rather than the stronger Sample Best. The main result therefore uses the direct matching counts and scores.

Two interpretations are not supported. First, score matching does not identify a unique causal transition in the original native trajectory: the alternate actions, boundary, output text, and intermediate states may differ. Second, exhaustive evaluation of a structured family does not establish its efficiency relative to another family of equal size. A same-count random-trajectory comparison would require candidates generated before reading rewards, under a fixed random protocol. Sampling from reward-filtered historical candidates would not provide that control.

The released dynamic winner rule prefers fewer switches among equal-scoring candidates. This affects which native representative is recorded. Where a one-switch alternative had already been visited and tied, that rule would favor it over a multi-switch candidate, but the winner-only manifest cannot establish the complete history of visited ties. We therefore keep the one-switch existence result separate from claims about the search’s internal discovery dynamics.

G EXECUTION VALIDITY AND REPRODUCTION BOUNDARIES

G.1 EXECUTION MODES AND THE UNRESOLVED PATH QUESTION

Evaluation	State and path	Checks or interpretive boundary
Fixed	Seven complete constant trajectories; OFF disables test-time writes.	Defines the per-example static oracle on the same continued-trained checkpoint.
Dynamic	Used-layer union from the complete candidate; unused layers use the standard MLP path.	Constant-result parity and forced winner capture are implemented; complete future-action invariance is not established.
Reverse	Replays the reversed schedule; preserves the native used-layer union.	Changes action-content alignment and state evolution; does not transplant stored native increments.
Reset/Deletion	Changes prescribed boundary weights while retaining native prefix cache and suffix actions.	Mixed-state replacements; a natural OFF-prefix continuation is a separate missing comparison.
K/P	Injects native boundary weights; changes cache donor and materialization path.	Boundary isolation is checked; subsequent weights are recomputed and may diverge.
R2	Exhausts exactly-one-switch candidates under the diagnostic evaluator.	Family exactness is not full-space optimality; path and repeated-execution uncertainties remain relevant.

Table 14: Execution-mode map. Checks implemented in the source are distinguished from new model-level validation, which this revision does not report.

The search evaluator derives its materialized layer set from the complete action sequence, including future actions. Layers not used anywhere are routed through the standard MLP; materialized layers can use chunkwise functional linear operations even before their first write. This source-level fact creates a numerical question: for identical input and action prefixes, can different future actions change prefix activations, cache entries, or effective weights through the selected implementation?

The code-level input-token and attention-mask parity check is narrower than this question. Equal inputs do not prove equal numerical computation across paths. Conversely, the fact that the paths differ is not proof that all dynamic gains are artifacts. Reverse preserves the used-layer union, and the search code checks fixed-seed predictions and replays its winner. These checks provide partial support while leaving the broader invariance question open.

The following validations are needed on the historical checkpoint, at its actual dtype and kernel settings, before stronger path-independent claims:

1. Preserve each schedule and materialization path while forcing proposed writes to zero; compare predictions and traces with the intended zero-write reference.
2. Compare standard and functional linear paths at zero offsets and at observed nonzero offsets, covering complete chunks, tails, generation entry, and later generated tokens.
3. Hold input and an action prefix fixed, vary only future actions, and check prefix hidden states, cache tensors, effective weights, and updates.
4. Verify apply-then-update at the same layer/chunk: accepting that chunk’s write must not change its already computed output.
5. Hold a complete trajectory fixed while changing only the scoring gold; its forward prediction and state must be invariant. Gold-dependent search selection is intentional and is a different test.
6. Repeat independent trajectory execution and report reward, prediction identity, logit distances, and actual weight differences, rather than relying only on nominal clip norms.

These are validation requirements, not completed results. If a check changes task outcomes, the affected comparisons need a clearly specified revised execution protocol and new results; new code semantics must not be combined with old scores without a parity demonstration.

G.2 WHAT CAN BE REPRODUCED FROM THIS PACKAGE

This revision packages the released, sanitized per-sample tables for sequential search, Reverse, Reset, Deletion, K/P, and R2; the 13-task fixed aggregates; and the 1,435 winning dynamic action sequences. The CPU script `analysis/recompute.py` validates sample IDs, static maxima, search eligibility, cohort relationships, factorial pairing, and enumeration counts. It generates the LaTeX table rows, figures, paired intervals, summary JSON, and provenance hashes used by this manuscript. It requires NumPy, pandas, and Matplotlib, not the trained model.

From the package root, run

```
python analysis/recompute.py
latexmk -pdf -interaction=nonstopmode -halt-on-error
main.tex
```

The PDF and generated tables are included, so reading or compiling the manuscript does not require rerunning the statistics. The recorded code revision is pinned in `analysis/provenance.json`; the original inference configuration is included separately. The analysis script is a new statistical reanalysis of historical outcomes, not a model-inference reproduction.

G.3 ASSETS AND EVIDENCE STILL MISSING

The public source snapshot excludes the historical model weights and raw benchmark inputs. The checkpoint and benchmark identity hashes are recorded, but a hash does not provide the corresponding artifact. A full inference replay also needs the tokenizer, per-task input text, answer records, sample-level generation budgets, and compatible hardware/software versions. The runtime versions recorded for this revision’s CPU analysis are not the historical GPU experiment environment.

Exact outer-training reconstruction is more limited. The historical 1.7B training recipe is not supplied, and the larger-model training configurations cannot safely replace it. The data reconstruction notes require historical anchors and describe a lost Stage2 selector/packer/RNG process. They replay an existing 10M-token artifact rather than recovering its original generation procedure. A new dataset or retraining run would be a new protocol, not a verified reconstruction of the reported checkpoint.

The MRQA sources include SQuAD and HotpotQA, which also appear as underlying QA families in evaluation. Shared source families do not demonstrate contamination, but excluding RULER itself from training-data construction does not rule out question, answer, or passage overlap. A complete audit requires the frozen source splits, IDs, normalized text hashes, and near-duplicate checks. No clean-overlap claim is made without that evidence.

The source includes engineering support for Qwen3-4B and Llama-3.1-8B, with settings that also change context length, chunk size, and action space. Those configurations are not completed formal robustness results and are not pooled with the 1.7B experiment. This manuscript consequently restricts its model and context claims to the historical 1.7B/16K evaluation.

Finally, matched-budget random Dynamic search, random permutations, complete natural-control continuations, same-size random R2 families, and norm-matched nonzero Deletion controls are absent from the supplied result artifacts. Their absence is reflected in the narrow claims of the main text. Publishing code makes definitions inspectable; it does not supply these missing measurements or establish end-to-end reproducibility.